
GramVest

SIH 2026 • PS 26091

M4 - Financial Engine, Scheme Rules, Viability Scoring & What-If

Module implementation handbook - finance / analytics owner

TEAM IMPLEMENTATION HANDBOOK

Prepared as a practical engineering guide for independent module development, stable integration, testing, and handoff.

Source basis: SIH 26091 Implementation Blueprint and the corresponding GramVest module
Markdown handbook.

Contents

1 1. Mission	1
2 2. Recommended tools	1
3 3. Repository	1
4 4. Financial flow	2
5 5. Project cost	2
6 6. Revenue model	2
7 7. Expense model	3
7.1 Variable	3
7.2 Fixed	3
7.3 Semi-variable	3
8 8. Operating surplus	3
9 9. EMI	3
10 10. Cash flow	3
11 11. P&L	4
12 12. Break-even	4
13 13. DSCR	4
14 14. Scheme rules	4
15 15. Scheme routing logic	5
16 16. Viability scoring	5
17 17. What-If Simulator	5
18 18. Stress scenarios	6
19 19. Unit testing	6
19.1 EMI	6
19.2 DSCR	6
19.3 Project cost	6
19.4 Scheme	7
19.5 Viability	7
19.6 What-If	7
20 20. Integration with M3	7
21 21. Integration with M5	7
22 22. Integration with M1	7
23 23. Definition of done	8
24 24. Common mistakes	8

1. Mission

Build the deterministic decision/calculation engine.

This is one of the highest-trust modules in GramVest.

The blueprint is explicit:

All numerical financial outputs must be produced by deterministic code. The LLM may explain the result but must never be the source of the calculation.

M4 owns:

- project cost
- promoter contribution
- financing requirement
- interest/tenure assumptions
- EMI/amortization
- moratorium handling
- revenue/expense projections
- working capital
- P&L
- cash flow
- break-even
- DSCR
- scheme rules
- viability scoring
- What-If recalculation

2. Recommended tools

Task	Tool
Language	Python
Numeric calculations	NumPy / Python decimal as appropriate
Tabular analysis	pandas
API implementation	FastAPI through M2
Testing	pytest
Notebook validation	Jupyter (for hand verification only)
DB policy data	PostgreSQL
DB inspection	DBeaver/pgAdmin
API testing	Postman
Version control	Git + GitHub

Do not let Jupyter become the production calculation engine. Production code must live in the back-end module.

3. Repository

```
backend/services/finance/  
├─ project_cost.py  
├─ financing.py  
├─ emi.py  
├─ cashflow.py  
├─ pnl.py  
├─ dscr.py  
├─ breakeven.py
```

└─ validation.py

backend/services/schemes/

└─ rules.py

└─ router.py

└─ versions.py

backend/services/scoring/

└─ opportunity_score.py

└─ viability.py

└─ risk_weights.py

backend/services/simulator/

└─ what_if.py

tests/finance/

tests/schemes/

tests/scoring/

tests/simulator/

4. Financial flow

Own Capital

→ Project Cost

→ Indicative Funding

→ Scheme Rule Check

→ EMI

→ Revenue/Expense

→ Cash Flow

→ DSCR

→ Feasibility Decision

This follows the blueprint exactly.

5. Project cost

Inputs may include:

- equipment
- setup
- initial inventory
- working capital
- other startup costs

Output:

project_cost

own_contribution

financing_required

Formula:

$\text{financing_required} = \max(\text{project_cost} - \text{own_contribution}, 0)$

6. Revenue model

A simple prototype may use:

$\text{monthly_revenue} = \text{units_sold} \times \text{selling_price}$

For a business with multiple products:

$\text{monthly_revenue} = \sum(\text{product_units} \times \text{product_price})$

Store assumptions.

7. Expense model

Separate:

Variable

- raw material
- packaging
- transport linked to sales

Fixed

- rent
- base salaries
- subscriptions
- maintenance

Semi-variable

- electricity
- diesel
- seasonal labor

This makes What-If analysis easier.

8. Operating surplus

`operating_surplus = monthly_revenue - monthly_operating_expenses`

Do not subtract loan payment twice.

Keep operating surplus and post-debt cash clearly separated.

9. EMI

Use standard amortization for normal term loans, with explicit handling for moratorium if required.

For monthly rate r , principal P , and total periods n :

$$\text{EMI} = P * r * (1+r)^n / ((1+r)^n - 1)$$

Handle the zero-rate case separately.

All assumptions must be recorded:

- principal
- annual rate
- payment frequency
- tenure
- moratorium method

10. Cash flow

Produce monthly or quarterly projections as required.

Example fields:

period
revenue
operating_expenses
operating_surplus

```
loan_payment
net_cash_flow
opening_cash
closing_cash
```

11. P&L

Keep P&L distinct from cash flow.

At minimum:

```
Revenue
- Operating Costs
= Operating Profit
- Interest/other financing costs where modeled
= Profit before tax/other modeled line items
```

For the prototype, clearly label the level of accounting detail provided.

12. Break-even

Use a transparent definition and document it in code/docs.

If using contribution-margin break-even:

```
break_even_units = fixed_cost / contribution_per_unit
```

Translate to months using monthly sales assumptions.

13. DSCR

Use a consistent documented definition.

A prototype may use a cash-flow available for debt service numerator and scheduled debt service denominator.

Example conceptual structure:

```
dscr = cash_available_for_debt_service / debt_service
```

Do not let the UI imply a government/bank-approved DSCR formula unless your source explicitly provides it.

14. Scheme rules

Store scheme policy data as versioned configuration, not hard-coded Python if statements.

Schema:

```
scheme
version
source
source_date
effective_date
project_cost_min
project_cost_max
funding_percent
max_funding
interest_rate
tenure_months
moratorium_months
eligibility_rules
notes
```

The blueprint gives example indicative tiers but explicitly warns that values are policy data and final eligibility belongs to the authority/bank. Use source/version metadata and label results as indicative.

15. Scheme routing logic

Flow:

```
user profile
+
project cost
+
own contribution
+
business attributes
+
configured scheme rules
→
possible routes
```

Return:

```
{
  "scheme_id": "...",
  "fit": "possible",
  "reasons": [],
  "limits": {},
  "assumptions": [],
  "source": "...",
  "version": "..."
}
```

Do not return "approved".

16. Viability scoring

The blueprint recommends this example policy:

```
25% Market Demand
20% Competition Opportunity
20% Capital Fit
20% Profit/Cash-Flow Potential
15% Risk Resilience
```

Keep weights configurable.

Inputs can be normalized to 0–100.

```
viability =
  0.25*market +
  0.20*competition +
  0.20*capital_fit +
  0.20*profit_potential +
  0.15*risk_resilience
```

Return component scores so the frontend can explain the score.

This is an example internal policy for the prototype, not an official government formula.

17. What-If Simulator

Input:

- selling price change
- customer/demand change
- raw material cost change
- operating expense change
- own capital change

- loan amount change

Example request:

```
{
  "base_scenario_id": "base-001",
  "changes": {
    "selling_price_pct": -10,
    "demand_pct": -20,
    "raw_material_cost_pct": 15,
    "operating_expense_pct": 0,
    "own_capital_change": 0,
    "loan_amount_change": 0
  }
}
```

Return:

```
{
  "baseline": {},
  "scenario": {},
  "delta": {},
  "risk_change": "...",
  "viability_change": -12,
  "assumptions": []
}
```

The blueprint says each scenario should be stored as a transparent set of assumptions.

18. Stress scenarios

Test at minimum:

- low capital
- project above scheme cap
- zero/negative margin
- high operating costs
- insufficient cash flow
- lower demand
- price decline

19. Unit testing

Create hand-verified test cases for:

EMI

- normal rate
- zero rate
- different tenure

DSCR

- healthy
- borderline
- insufficient cash flow

Project cost

- exact match
- capital greater than cost
- capital lower than cost

Scheme

- below minimum
- within range
- above cap

Viability

- all 100
- all 0
- mixed components

What-If

- no change
- demand -20%
- cost +15%
- combined shocks

20. Integration with M3

M3 gives:

- market score
- competition score
- demand/customer estimates
- affordability evidence
- market gap evidence

M4 uses these to produce viability.

21. Integration with M5

M4 sends:

- verified financial values
- scenario outputs
- scheme route + version
- viability score/components

M5 may explain these values but may not recalculate them.

22. Integration with M1

M1 expects:

- project cost
- own contribution
- financing gap
- EMI
- cash flow
- P&L
- DSCR
- break-even
- viability
- scheme route
- scenario results

All response fields must be documented.

23. Definition of done

M4 is done when:

- deterministic calculations work
- tests pass
- scheme rules are versioned
- score components are explainable
- What-If works
- API schemas are stable
- no LLM is used for numerical calculation
- edge cases are tested

24. Common mistakes

- doing finance in frontend
- hard-coding scheme rules
- changing formula definitions without documentation
- mixing monthly and annual rates
- forgetting moratorium assumptions
- returning unexplained score numbers
- using stale policy values without source/version metadata